SafeRouteAI

Higher National Diploma in Software Engineering

Final Project Documentation

2025.1F

School of Computing and Engineering

National Institute of Business Management

Colombo-7

Academic Year 2025

SafeRouteAI

SUBMITTED BY

COHNDSE251F-012 - M.K.W.A.D.S.S. Siriwardhana
COHNDSE251F-017 - I. N.Y. Dias
COHNDSE251F-043 - G.N.A. Kodagoda
COHNDSE251F-074 - H.M.D.P. Herath

Higher National Diploma in Software Engineering
School of Computing and Engineering
National Institute of Business Management
Colombo-7

24/02/2026

"The project is submitted in partial fulfillment of the requirement of the Higher National Diploma in Software Engineering HNDSE25.1F of National Institute of Business Management."

Declaration

"I certify that this project does not incorporate without acknowledgement, any material previously submitted for a HND in any institution and to the best of my knowledge and belief, it does not contain any material previously published or written by another person or myself except where due reference is made in the text. I also hereby give consent for my project report, if accepted, to be made available for photocopying and for interlibrary loans, and for the title and summary to be made available to outside organizations."

Date: ……

Signatures:

……	……
M.K.W.A.D.S.S. Siriwardhana	I. N.Y. Dias
……	……
G.N.A. Kodagoda	H.M.D.P. Herath

Signature of the Supervisor: ……

Mr. Sujeewa Sampath

Abstract

SafeRoute AI is a cognitive and multi-platform mobile application designed to bridge the alarming road safety gap on Sri Lankan roads. This application has been created using React Native (Expo), along with TypeScript. This application incorporates GPS navigation, accelerometer-based pothole detection on the device, driving behaviour analytics, automated response for crashes, and community-based hazard mapping.

This application utilizes the Firebase Realtime Database for cloud-based synchronisation, the Google Maps Platform for GPS navigation, and Android sensor APIs for sub-second hazard detection and notification. Emergency notifications are sent to pre-registered individuals via the Telegram API for live location sharing during a crash.

This application is implemented through three interfaces: the Driver application for real-time road monitoring and safety scoring, the Admin application for high-level application management and pothole management, and the Outsource application implemented using React and Leaflet for high-level hazard mapping. This application has been tested for its feasibility on Sri Lankan roads and its impact on road safety for drivers.

Keywords, Abbreviations, Tables & Figures

Keywords

Road Safety, Pothole Detection, Accelerometer, GPS Navigation, Firebase Realtime Database, React Native, Expo, Driving Score, Crash Detection, Sri Lanka, Mobile Application, Crowdsourcing, Real-Time Alerts, Geolocation, High-Pass Filter, TypeScript, Sensor Fusion, Emergency SOS, Telegram API, Leaflet.js, Web Dashboard.

Abbreviations

Abbreviation	Full Form
API	Application Programming Interface
ETA	Estimated Time of Arrival
GPS	Global Positioning System
G-Force	Gravitational Force — unit of acceleration (9.81 m/s^2 = 1G)
GIS	Geographic Information System
IoT	Internet of Things
SOS	Save Our Souls (Emergency Signal)
TTS	Text-to-Speech
UAT	User Acceptance Testing
UI/UX	User Interface / User Experience
UUID	Universally Unique Identifier
SDK	Software Development Kit
NoSQL	Non-relational Structured Query Language (database type)
RDA	Road Development Authority (Sri Lanka)

Table 1 - Abbreviation

List of Figures

- Figure 1 — Use Case Diagram
- Figure 2 — ER Diagram
- Figure 3 — User Interfaces
- Figure 4 — Admin Interfaces
- Figure 5 — Project Gantt Chart

List of Tables

- Table 1 — Abbreviations List
- Table 2 — Software Development Tools & Technologies
- Table 3 — Safety Score Engine Penalty Schema
- Table 4 — Problem-Solution Mapping
- Table 5 — Pothole Lifecycle
- Table 6 — Firebase Data Structure for Voting
- Table 7 — Driver Use Case Specifications
- Table 8 — Administrator Use Case Specifications
- Table 9 — Outsource Web Dashboard Use Case Specifications
- Table 10 — ER Diagram Entities & Relationships
- Table 11 — System Testing Results Summary

Contents

Chapter 1: Introduction 9
1.1 Introduction of the Domain 9
1.2 Research Gap 10
1.3 Research Problem Definition 10
1.4 Main & Specific Objectives 11
1.5 Proposed Solution 12
1.6 Chapter Summary 13
Chapter 2: Methodology 14
2.1 Data Collection Methods 14
2.2 Software Process Model 15
2.3 Software Development Tools 16
2.4 Testing Strategies 17
2.5 Implementation Plan 18
2.6 Chapter Summary 18
Chapter 3: System Architecture & Implementation 19
3.1 Overall System Architecture 19
3.2 Sensor Fusion & Pothole Detection Algorithm 19
3.3 GPS-Based Navigation & Safety Monitoring 21
3.4 Safety Score Engine 22
3.5 Firebase Real-Time Database Integration 22
3.6 User Interface Implementation 23
Chapter 4: System Analysis & Design 25
4.1 Problem and Solution Analysis 25
4.2 Smart Pothole Voting System Design 25
4.3 Use Case Specification 29
4.4 Data Management 24
4.5 UI Design 27
4.6 Chapter Summary 40
Chapter 5: Testing & Results 30
5.1 Testing Overview 41
5.2 Unit Testing Results 41
5.3 Integration Testing Results 42
5.4 System Testing Results 42
5.5 User Acceptance Testing (UAT) 43

5.6 Chapter Summary 44
Chapter 6: Project Timeline & Gantt Chart 45
6.1 Implementation Schedule 45
6.2 Key Milestones 45
Chapter 7: Conclusion & Future Work 46
7.1 Conclusion 46
7.2 Limitations 46
7.3 Future Work 47
Chapter 8:References 48

Chapter 1: Introduction

1.1 Introduction of the Domain

Road safety remains one of the most pressing public health challenges in developing nations. Sri Lanka's Road network spans over 100,000 kilometers, yet a substantial proportion of these roads suffer from chronic underfunding and deferred maintenance, resulting in widespread potholes, uneven surfaces, and unpredictable driving conditions. According to the Sri Lanka Police Traffic Headquarters, road accidents claim over 3,000 lives annually, with vehicle damage attributable to poor road surface conditions contributing significantly to economic losses, driver distress, and premature vehicle degradation.

The rapid proliferation of smartphones now ubiquitous across Sri Lanka, including rural areas combined with advances in mobile sensor technology, presents an unprecedented opportunity to construct intelligent, community-driven road safety solutions without requiring dedicated road infrastructure investment. Contemporary smartphones are equipped with high-precision GPS receivers, three-axis MEMS accelerometers, gyroscopes, and persistent internet connectivity. Harnessing these embedded capabilities allows developers to create applications that proactively identify road hazards, deliver real-time navigation guidance, monitor driving behavior, and coordinate emergency response, all from within a single consumer device.

SafeRoute AI directly exploits these capabilities. Built on the React Native (Expo) framework with TypeScript, it integrates Google Maps Platform APIs, Firebase Realtime Database, on-device accelerometer sensing, and GPS tracking to deliver an end-to-end road safety companion. The platform addresses navigation, hazard detection, crash response, driver scoring, and community data crowdsourcing within a single, cohesive user experience tailored specifically to the Sri Lankan Road context.

1.2 Research Gap

Despite the widespread availability of commercial navigation applications such as Google Maps and Waze, a significant gap exists in purpose-built road safety tools for the Sri Lankan context:

- Existing navigation applications lack proactive, sensor-based pothole detection that automatically reports hazards to a shared, persistent community database accessible to all users in real time.
- No mainstream application provides automated crash detection coupled with an emergency SOS protocol specifically designed for Sri Lanka's local emergency services infrastructure.
- Driver behavior analytics specifically harsh braking, sudden acceleration, and speed consistency metrics are absent from all locally available mobile navigation applications.
- Crowdsourced road condition data from Sri Lankan roads is sparse, fragmented, and not integrated into any real-time, openly accessible database maintained at the community level.
- Voice-guided, hands-free safety alerts remain largely unavailable in local applications, increasing driver cognitive load and distraction risk.

This research gap motivated the development of SafeRoute AI as a holistic, technically rigorous solution that addresses each of these deficiencies through a single, integrated mobile platform.

1.3 Research Problem Definition

The central research problem addressed by this project is:

> *"How can a mobile application leverage on-device sensors, real-time GPS tracking, and cloud-connected community data to proactively detect, report, and warn drivers of road hazards, while simultaneously monitoring driving behavior and enabling rapid emergency response — specifically within the Sri Lankan Road network context?"*

1.4 Main & Specific Objectives

Main Objective

To design, develop, and validate a mobile application that measurably improves driver safety in Sri Lanka by combining real-time pothole detection, intelligent navigation, driving behavior analytics, crash detection, and community-driven hazard mapping into a single, accessible, and deployable platform.

Specific Objectives

1. Implement sensor-based pothole detection using the Android accelerometer with a high-pass filter algorithm (ALPHA = 0.8) to eliminate gravitational noise and accurately identify road surface impacts exceeding 3.0 m/s^2 linear acceleration.
2. Develop a GPS-powered navigation module restricted to the Sri Lankan Road network, providing turn-by-turn visual guidance, real-time route rendering via the Google Maps Directions API, and live speed monitoring.
3. Integrate Firebase Realtime Database for bidirectional, crowd-sourced pothole data synchronization with sub-second propagation across all active users.
4. Design a driving behavior analytics engine that tracks harsh braking events (delta speed < -3.5 m/s over 1s), speed violations above 80 km/h, and acceleration patterns, computing a cumulative 0-100 safety score per trip.
5. Implement a crash detection module triggered at impact forces exceeding 8.0G, automatically activating an SOS alert to pre-registered emergency contacts via Telegram Bot API with live GPS location.
6. Develop a voice-guided alert system using Expo Speech TTS to deliver hands-free safety warnings, reducing driver visual distraction.
7. Build a community module displaying a shared pothole map and driver leaderboard ranked by cumulative safety score, incentivizing safer driving behavior.
8. Develop an Admin Mobile App with system overview, pothole management, and user management features connected to the same firebase backend.
9. Develop an Outsource Web Dashboard with React and Leaflet.js for spatial hazard visualization, real-time syncing of data, and deletion of potholes for external stakeholders.

1.5 Proposed Solution

SafeRoute AI is realized as a cross-platform mobile application targeting Android — the dominant platform in Sri Lanka — built on the React Native (Expo SDK) framework. The application architecture is organized into five tightly integrated core modules:

- Navigation & Intelligent Road Preview Module: The Google Maps Platform with Sri Lanka-specific constraints (country: lk) to enable accurate search, live route visualization with polylines (blue, 5px stroke), and live directions. The Intelligent Road Preview Module utilizes pothole data from Firebase and weather information from the OpenWeather API to provide live safety risk assessment information for the user's route.
- Pothole Detection Module: Android-optimized accelerometer sensing with a two-stage low-pass/high-pass gravity filter running at 10 Hz (100 ms interval). Detected impacts automatically persisted to Firebase with GPS coordinates, bump force magnitude, and timestamp.
- Safety Monitoring Module: Continuous GPS speed tracking (clamped from Android's potentially negative speed returns), harsh braking detection using inter-sample speed deltas, and per-trip safety score calculation with a 5-second shared cooldown to prevent alert flooding.
- Crash Detection & SOS Module: High-G impact threshold monitoring (8.0G), automated Telegram Bot API notifications to registered emergency contacts with live location coordinates.
- Community & Gamification Module: Shared pothole database visible as orange map markers to all active users, and a driver leaderboard ranked by cumulative safety score.
- Admin Mobile App: Real-time system overview, pothole management with filtering and resolution, and user management interface.
- Outsource Web Dashboard: React + Leaflet.js interactive map plotting of all detected potholes from Firebase, live sync, hazard inspection, statistics, delete, etc.

All real-time data is synchronized through Firebase Realtime Database, ensuring that hazard information detected by any driver is immediately available to all active users. Haptic vibration feedback (Android Vibration API) complements visual dashboard elements and banner alerts to create a multi-modal, distraction-minimizing safety experience.

1.6 Chapter Summary

In this chapter, the domain environment for SafeRoute AI was set within the road safety scenario of Sri Lanka, the research gap was identified for all three interfaces of the system, the research problem was formally stated, and the overall and specific objectives of the research were outlined. The proposed three interfaces of the system were also outlined, showing how it addresses each of the research gaps.

Chapter 2: Methodology

2.1 Data Collection Methods

The project employed a mixed-methods data collection strategy combining primary, secondary, and crowdsourced data sources:

- Primary Data: Real-world accelerometer readings collected during controlled test drives on diverse Sri Lankan Road types (urban Colombo streets, rural B-roads, and highway sections) to empirically calibrate the bump detection threshold of 3.0 m/s^2 and validate the high-pass filter coefficient ALPHA = 0.8. GPS speed data was logged and validated against posted speed limits.
- Secondary Data: Publicly available road condition reports from the Road Development Authority (RDA) of Sri Lanka, peer-reviewed academic literature on MEMS accelerometer-based pothole detection algorithms, Firebase Realtime Database documentation, and Expo SDK sensor API specifications.
- Crowdsourced Data: Pothole GPS coordinates and bump force readings submitted automatically by the application during real user sessions, aggregated in real time within the Firebase Realtime Database /detected potholes node.
- User Feedback: Structured questionnaires distributed to beta testers following supervised test drives to validate detection accuracy, navigation usability, and safety score relevance.

2.2 Software Process Model

SafeRoute AI was developed using the Agile (Scrum) methodology, delivered across four two-week sprints with a total project duration of twelve weeks. This iterative approach was selected for the following technical and project management reasons:

- Frequent deliverable cycles enabled early detection and correction of sensor calibration errors on physical devices before they propagated to subsequent modules.
- Sprint retrospectives provided structured opportunities to refine the pothole detection algorithm threshold based on real-world field test data gathered during each sprint.
- Backlog management allowed features such as voice alerts, the leaderboard, and the trip summary modal to be prioritized and delivered incrementally without blocking core navigation functionality.
- Continuous integration via Git/GitHub ensured that all team members always maintained a working build, reducing integration risk.

The four major sprints progressed as follows:

- Sprint 1 - Navigation & Map Integration: Google Maps and Places API integration for the Driver App.
- Sprint 2 - Sensor-Based Detection & Firebase Sync: Accelerometer Filter, Pothole Reporting, Firebase Synchronization.
- Sprint 3 - Safety Scoring, Crash Detection & Telegram SOS: Safety Engine, Braking Detection, Telegram Bot API for SOS.
- Sprint 4 - Admin App, Web Dashboard, Voice Alerts & UAT: Admin Mobile App, Leaflet.js-based Dashboard, Voice Alerts, Leaderboard, UAT.

2.3 Software Development Tools

Category	Technology / Tool	Purpose
Frontend Framework	React Native (Expo SDK 52)	Cross-platform Android UI development
Language	TypeScript	Type-safe application logic and component props
Navigation & Maps	Google Maps Platform	Map rendering, Directions API, Places API
Backend / Database	Firebase Realtime Database	Cloud data sync, pothole storage, real-time listeners
Sensors	Expo Sensors (Accelerometer)	Pothole detection & crash detection
Location Services	expo-location	GPS tracking, speed measurement, coordinates
Voice Alerts	Expo Speech (TTS)	Hands-free audio safety warnings
State Management	React Hooks (useState, useRef)	UI state and closure-safe mutable refs
Distance Calculation	geolib	Haversine GPS distance computation for proximity alerts
UI Icons	Ionicons, MaterialCommunityIcons	Navigation and status iconography
IDE	Visual Studio Code	Primary development environment
Version Control	Git / GitHub	Source code management and team collaboration
Testing	Jest + Expo DevTools	Unit testing and device debug tooling

Table 2 – Software Development Tools & Technologies

2.4 Testing Strategies

Unit Testing

Individual functions — including the accelerometer high-pass filter computation, linear acceleration magnitude calculation, speed clamping logic, harsh braking delta computation, and Firebase read/write operations — were tested in isolation using Jest. Mock sensor data arrays simulating bump events (magnitude = 4.2 m/s^2) and smooth road conditions (magnitude < 1.5 m/s^2) were used to validate threshold behavior without requiring physical device access.

Integration Testing

The interaction between the GPS location watcher, pothole detection system, Firebase synchronization, and alert trigger chain was validated as an integrated subsystem. Test scenarios included simultaneous sensor reads at 10 Hz alongside GPS location updates arriving every 2 meters, verifying that the shared 5-second cooldown correctly prevented overlapping alerts.

System Testing

End-to-end system testing was performed on physical Android devices (Samsung Galaxy A series and Redmi Note series) across multiple road types in the Western Province of Sri Lanka. GPS accuracy (sub-5m CEP), pothole detection sensitivity, alert timing, and Firebase sync latency (measured at under 800ms on 4G networks) were systematically evaluated and documented.

User Acceptance Testing (UAT)

A cohort of 12 drivers conducted supervised test drives across pre-selected routes in Colombo and Negombo. Feedback was collected via structured questionnaires evaluating alert relevance (94% positive response), navigation accuracy, UI clarity, and safety score perceived fairness. Minor UI adjustments were incorporated following UAT findings.

2.5 Implementation Plan

The implementation proceeded across four Agile sprints spanning 12 weeks as detailed in Chapter 4. Key sprint outcomes were:

- Sprint 1 (Weeks 1-3): Google Maps integration with Sri Lanka-constrained Places autocompletes, real-time route polyline rendering via MapViewDirections, and ETA/distance calculation from the Directions API response.
- Sprint 2 (Weeks 4-6): Accelerometer high-pass filter with seeded gravity baseline, Firebase pothole auto-reporting on bump detection, GPS proximity alert system using geolib Haversine distance, and Android vibration confirmation feedback.
- Sprint 3 (Weeks 7-9): Continuous speed monitoring with Android -1 clamping, harsh braking delta detection at -3.5 m/s threshold, per-trip 0-100 safety score with weighted penalty system, and crash detection stub at 8.0G.
- Sprint 4 (Weeks 10-12): Expo Speech TTS voice alert integration, community leaderboard module, trip summary modal with distance/duration/score analytics, full UI polish, and formal UAT execution.

2.6 Chapter Summary

This chapter outlined the mixed-methods data collection approach used to calibrate and validate SafeRoute AI's sensor algorithms, justified the selection of Agile Scrum as the software process model, catalogued the complete development tools and technology stack, and defined the four-tier testing strategy. The 12-week sprint implementation plan provided a structured roadmap that successfully delivered all seven specific objectives within budget and schedule.

Chapter 3: System Architecture & Implementation

3.1 Overall System Architecture

SafeRoute AI follows a client-cloud architecture where the React Native mobile client handles all sensing, user interaction, and local state management, while Firebase Realtime Database serves as the shared, persistent cloud backend. The architecture eliminates the need for a dedicated application server by leveraging Firebase's real-time synchronization capabilities directly from the mobile client using the Firebase JavaScript SDK.

The system can be conceptually divided into three horizontal layers:

- Presentation Layer: React Native UI components (MapView, SafeAreaView, Modal, Animated) rendered on the Android devices; React + Leaflet.js rendered in a web browser for the Outsource Dashboard.
- Business Logic Layer: Sensor processing (Accelerometer high-pass filter), GPS analysis (speed monitoring, proximity detection, braking calculation), safety score engine, Telegram SOS dispatch and alert management, implemented as TypeScript functions and React effects within App.tsx.
- Data Layer: Firebase Realtime Database providing persistent storage of pothole records, and the Google Maps Platform providing routing, geocoding, and place search services.

3.2 Sensor Fusion & Pothole Detection Algorithm

The Android Accelerometer Challenge

A critical platform-specific engineering challenge is that Android accelerometers report raw values inclusive of gravitational acceleration (~9.81 m/s^2), unlike iOS which provides gravity-compensated linear acceleration. A naive magnitude checks on raw values would produce a constant reading of approximately 9.8 m/s^2 at rest, rendering threshold-based bump detection impossible.

SafeRoute AI resolves this through a two-stage digital filter implementation:

Stage 1: Low-Pass Filter (Gravity Estimation)

A recursive exponential moving average filter isolates the slow-changing gravity vector from the composite raw accelerometer reading. The filter is defined by the recurrence relation:

```
gravity[t] = ALPHA * gravity[t-1] + (1 - ALPHA) * raw[t]
where ALPHA = 0.8
```

where ALPHA = 0.8 is the filter coefficient. A higher ALPHA value increases the gravity estimate's inertia, making it slower to respond to genuine acceleration events and thus better at separating the static gravity component from dynamic bumps. The filter is seeded on the first sensor reading to prevent a cold-start artefact.

Stage 2: High-Pass Filter (Linear Acceleration Extraction)

Once gravity is estimated, the linear (non-gravitational) acceleration is derived by component-wise subtraction:

```
linear[t] = raw[t] - gravity[t]
```

The Euclidean magnitude of the three-dimensional linear acceleration vector is then computed:

```
bump = sqrt (linX² + linY² + linZ²)
```

A pothole event is triggered when bump > 3.0 m/s^2 — a threshold empirically validated through field testing on Sri Lankan roads — provided the 5-second cooldown has elapsed since the last alert.

On-Device Pothole Reporting

Upon pothole detection, the following actions occur synchronously:

1. The current GPS coordinates from currentLocRef are bundled with the timestamp, bump force magnitude, and detection source ('accelerometer') into a Firebase push operation to /detected_potholes.
2. Android haptic vibration feedback is triggered (pattern: [0, 80, 60, 80] — double pulse).
3. The safety score is decremented by 10 points, and a banner alert is rendered via the Animated API with a 250ms fade-in and 400ms fade-out.

3.3 GPS-Based Navigation & Safety Monitoring

Navigation Module

The navigation module uses the react-native-maps MapView component with PROVIDER_GOOGLE, configured with a custom dark map style (24 JSON style rules) optimized for high-contrast road visibility in daytime and low-light conditions. Route rendering is handled by the react-native-maps-directions MapViewDirections component, which invokes the Google Maps Directions API and renders the resulting polyline as a 5px blue stroke.

Destination search is provided by the react-native-google-places-autocomplete component, constrained to 'country: lk' to prevent off-route international results, and biased toward the user's current GPS position with a 50km radius.

Real-Time Speed Monitoring

Speed is retrieved from loc.coords.speed in the expo-location watchPositionAsync callback, triggered every 2 metres of movement (distanceInterval: 2) or every 1000ms (timeInterval: 1000), whichever occurs first. The Android platform can return -1 for speed when measurement is unavailable; the application clamps this with Math.max (0, speed) before conversion to km/h.

When speed exceeds 80 km/h, a persistent alert banner is displayed with the live speed reading in red. The speed indicator in the dashboard stat box also changes from white to red above this threshold.

Harsh Braking Detection

Inter-sample speed delta is computed on every GPS location update. A harsh braking event is flagged when the speed decreases by more than 3.5 m/s within a single 1-second GPS update interval, equivalent to approximately 0.36G deceleration. A minimum previous speed of 2 m/s is enforced to suppress false positives from GPS jitter at near-standstill. Detected events decrement the safety score by 5 points with a 3-second independent cooldown.

GPS Proximity Pothole Alert

On each GPS location update, the full potholeRef array (maintained as a closure-safe ref mirroring Firebase state) is scanned using geolib.getDistance() to find recorded potholes within 100m of the driver's current position, excluding those within 5m (already passed). The nearest qualifying pothole triggers a proximity alert banner and haptic pulse, using the same 5-second shared cooldown as the accelerometer path to prevent overlapping alerts.

3.4 Safety Score Engine

Each trip begins with a perfect score of 100 points. The scoring engine applies the following penalty scheme during the trip:

Event Type	Penalty	Cooldown
Pothole detected by accelerometer (bump > 3.0 m/s²)	-10 points	5 seconds (shared)
Harsh braking event (delta speed < -3.5 m/s in 1s)	-5 points	3 seconds (independent)
Speed violation (> 80 km/h)	Alert only	No penalty — continuous alert while active
Crash detection (> 8.0G)	SOS triggered	Trip ends / emergency mode

Table 3 –Safety Score Engine Penalty Schema

The score is clamped at a minimum of 0 to prevent negative values. Upon trip completion, the score is classified as: 80-100 (Excellent Driver — green), 50-79 (Drive More Carefully — amber), or 0-49 (Dangerous Driving Detected — red), displayed in the Trip Summary Modal.

3.5 Firebase Real-Time Database Integration

Firebase Realtime Database was selected over Firestore for this project due to its superior support for live, low-latency data streaming via the onValue listener, which pushes pothole updates to all connected clients within milliseconds of a new detection event being committed. The Firebase JavaScript SDK (v9 modular API) is used with the following data operations:

- Read (onValue listener): Subscribed on application mount to the /detected_potholes node; triggers a full state refresh whenever any client adds a new record. The parsed array is stored in both the potholes React state (for UI rendering of markers) and potholeRef (for closure-safe access within GPS callbacks).
- Write (push): On each detected pothole event, a new record is pushed to /detected_potholes with auto-generated key, containing latitude, longitude, detectedBy, bumpForce, and timestamp fields.

- Update (set/update): Admin resolution writes set status to 'resolved'. Accident records written to /accidents node on SOS trigger.

The Firebase API key embedded in the application is the project-specific Google Maps API key (GOOGLE_MAPS_API_KEY) also used for Maps, Directions, and Places services, consistent across all Google Cloud Platform services bound to the same project.

3.6 Telegram Bot API Integration for SOS

Emergency SOS messages are sent via Telegram Bot API instead of SMS, which supports rich messages with live GPS coordinates. When a crash event greater than 8.0G is detected and confirmed:

- The Telegram Bot API sendMessage endpoint is invoked with the user's name, accident time, and GPS coordinates encoded as a Google Maps link.
- Pre-configured emergency contacts with Telegram Chat IDs will be notified immediately.
- Accident data is stored in the Firebase /accidents node for admin monitoring.

3.7 Outsource Web Dashboard

The Outsource Web Dashboard is a React web app that connects to the same Realtime Database instance. It is built with Leaflet.js and offers:

Interactive dark-mode map with all records from /detected_potholes plotted as interactive markers.

- Markers with pop-ups containing bump force, detecting user, time, and current status.
- Synced live data. The map and statistics will update in real time with new pothole reports.
- Side statistics cards for total potholes, active count, and resolved count.
- Delete and resolve function for outsource operators to delete invalid hazard markers.

3.8 Admin Mobile App Architecture

Admin Mobile App: This is a separate React Native app that uses the same Firebase backend. It has the following features:

- System Overview Dashboard: This gives a summary of the system statistics: total users, total trips, active potholes, and blocked accounts.
- Pothole Management: This allows the user to view all potholes by filtering them by "Active", "Resolved", and "All".
- Mark a pothole as "Resolved".
- View the exact location via an external map link.
- User Management: This allows the user to view and manage all registered driver accounts.
- Trip Analytics: This gives the user access to system-wide driving activity statistics.

3.9 User Interface Implementation

The application UI is built entirely in React Native with StyleSheet-defined styles. The dark slate colour palette (#0f172a background, #1e293b card surfaces) was chosen for optimal map contrast and reduced glare in outdoor driving conditions. Key UI components include:

- MapView: Full-screen absolute fill with custom 24-rule dark map style. Pothole markers rendered as orange Marker components. Destination rendered as a red Marker. Active route rendered as a blue MapViewDirections polyline.
- Search Box (pre-trip): GooglePlacesAutocomplete with rounded input (52px height) positioned at top: 50 with elevation: 8 shadows on Android.
- Navigation Header (active trip): Slide-in panel displaying distance remaining and live speed badge, colour-coded red above 80 km/h.
- Alert Banner: Animated.View with fade opacity, amber left border, and Ionicons warning icon. Renders above the map at z-index 20.
- Dashboard: Bottom-anchored card with stats grid (Speed, Limit, Score), detection mode badge, pothole count bar, SOS button, and End Trip button.
- Trip Summary Modal: Full-screen slide-up overlay with score circle (colour-coded border), verdict text, and 2x2 stats grid (Distance, Duration, Potholes, Top Speed).

Chapter 4: System Analysis & Design

4.1 Problem and Solution Analysis

Problem	Proposed Solution	Module
1. Potholes cause vehicle damage but are not reported in real time.	Accelerometer-based detection with automatic Firebase reporting.	Pothole Detection Module
2. A single false report can permanently pollute the hazard map.	Community voting system: pothole added on 1 detection, automatically removed when 2 different users pass without triggering a bump.	Smart Pothole Voting Engine
3. Drivers have no advance warning of nearby hazards.	GPS proximity alert triggered 100m before any active pothole using geolib Haversine distance.	Proactive Alert System
4. Crash events go undetected with no emergency response.	8.0G threshold crash detection with automatic SOS to emergency contacts.	Crash Detection & SOS
5. Drivers cannot assess their own safety behavior.	Per-trip safety score based on braking, speed, and pothole events.	Safety Scoring Engine
6. Mobile phone navigation increases driver distraction.	Multi-modal alerts: voice TTS, haptic vibration, and ambient dashboard indicators.	Voice Alert & Haptic System
7. No incentive mechanism for safer driving.	Competitive leaderboard ranking drivers by cumulative safety score.	Gamification Module
8. Admin has no visibility over user accounts or system health.	Admin dashboard: Manage Users, View Pothole Map, View Analytics, View Leaderboard.	Admin Dashboard Module

Table 4 – Problem Solution Mapping

4.2 Smart Pothole Voting System Design

A fundamental challenge in any community-driven hazard mapping system is data quality. A single false detection — caused by a speed bump, railway crossing, or car door slam — can create a phantom pothole marker that persists indefinitely, eroding user trust. Conversely, manually removing potholes requires admin intervention that does not scale with a large user base.

SafeRoute AI addresses this through an automated community voting system that governs the entire lifecycle of every pothole record — from creation to automatic removal — without any manual administrator involvement.

Pothole Lifecycle: The Voting Algorithm

Event	**System Action**
CREATION — 1 user triggers bump > 3.0 m/s²	A new pothole record is pushed to /detected_potholes with status: 'active', reporters: [user_id], clean_passes: 0, clean_passers: []. The orange marker appears on the map for all users immediately.
CONFIRMATION — subsequent users also detect a bump at same location (within 15m radius)	The existing record's reporters array is updated to include the new user_id (if not already present). Each additional confirmation strengthens the record's credibility score.
CLEAN PASS — a user drives through the pothole zone (within 15m) without triggering a bump	The system checks: (a) the driver has not previously been in the reporters list, and (b) they have not already been counted as a clean_passer. If both conditions pass, their user_id is appended to clean_passers[] and clean_passes is incremented by 1.
AUTO-REMOVAL — clean_passes reaches 2	Once 2 distinct users have passed the location without any bump detection, the system automatically updates the record status from 'active' to 'resolved'. The orange map marker is removed from all connected clients in real time via the Firebase onValue listener. No admin action is required.

Table 5 – Pothole Lifecycle

Anti-Gaming Rules

The voting system includes safeguards to prevent a single user from artificially manipulating pothole lifecycle:

- A user who originally reported a pothole (appears in reporters []) cannot be counted as a clean_passer for that same record. This prevents the reporter from immediately 'undoing' their own report.
- A user can only contribute one clean pass per pothole record, regardless of how many times they drive through the location. This prevents a single driver from auto-removing a pothole by driving through it twice.
- The initial reporter's user_id is recorded in reporters []. Any further bump detections from different users at the same location (within 15m) increment a confirmation_count field, providing an indication of pothole severity.
- If a pothole was auto-resolved but a new bump is detected at the same GPS coordinates later, a brand-new pothole record is created — the resolved record is not re-activated. This accommodates roads that are temporarily patched but deteriorate again.

Firebase Data Structure for Voting

The following shows the structure of a single pothole record in /detected_potholes that implements the full voting system:

Field	Type	Purpose in Voting System
pothole_id	string (auto-key)	Unique Firebase push key identifying this pothole record.
latitude	number	GPS latitude of the detected impact location.
longitude	number	GPS longitude of the detected impact location.
bump_force	string (m/s^2)	Linear acceleration magnitude at time of detection (e.g. '4.52').
detected_by	string	'accelerometer' — the source of the original detection.
timestamp	number	Unix timestamp (ms) of the original detection event.
status	string	'active' when visible on map. Set to 'resolved' when auto-removal triggers. Map filter shows only status='active' markers.

reporters	string[] (array)	Array of user_ids who have detected a bump at this location. Used to block reporters from acting as clean_passers.
confirmation_count	number	Number of distinct users (beyond the original) who also hit a bump here. Indicates pothole severity.
clean_passes	number	Counter of distinct users who passed this location without a bump. Auto-removal fires when this reaches 2.
clean_passers	string[] (array)	Array of user_ids who have contributed a clean pass. Prevents a single user contributing more than one clean pass.

Table 6 – Firebase Data Structure for Voting

Clean Pass Detection Logic

A 'clean pass' is registered in the GPS location watcher callback when all of the following conditions are simultaneously true:

4. The driver's current GPS coordinates are within 15 metres of a pothole record (computed using geolib.getDistance()).
5. The pothole's status is 'active' (resolved records are skipped).
6. The driver's user_id is NOT present in the pothole's reporters[] array — confirming they are not the original reporter.
7. The driver's user_id is NOT already in the clean_passers[] array — confirming this is their first clean pass for this specific record.
8. No bump event has been detected in the current 5-second cooldown window at this location (confirming the pass was genuinely smooth).

When all five conditions are met, Firebase is updated atomically: the driver's user_id is appended to clean_passers[], clean_passes is incremented by 1, and if clean_passes then equals 2, status is set to 'resolved'. The map removes the orange marker within milliseconds for all connected clients.

4.3 Use Case Specification

System Actors

- Primary Actor 1- Driver (User): The individual using the mobile application on an Android device while navigating Sri Lankan roads.
- Primary Actor 2 - System Administrator (Admin): A privileged back-office user who manages platform data through a dedicated admin dashboard connected to the same Firebase backend.
- Primary Actor 3- Outsource: An external stakeholder accessing the web-based dashboard for spatial hazard monitoring.
- Secondary Actors: Google Maps Platform; Firebase Realtime Database; Expo Sensors API; Telegram Bot API.

Driver Use Cases

Use Case-Driver	Description
User Authentication	User registers or logs in. Includes a Validate sub-process for credential verification via Firebase Authentication.
Search Destination	User searches and selects a destination via Google Places Autocomplete, constrained to Sri Lanka (country:lk).
Start Navigation	Initiates the journey. Route polyline rendered via MapViewDirections. Navigation header shows distance remaining and live speed.
Receive Voice Guidance	Delivers hands-free navigation and safety warnings via Expo Speech TTS engine.
View Driving Pattern	Continuously tracks GPS speed, harsh braking events, and speed violations. Extends to View Driver Score.
Receive Alerts	Extends to: Speed Warning Alert (> 80 km/h), Proactive Hazard Alert (within 100m of pothole), and SOS Alerts (crash detected).
Detect Potholes	Accelerometer high-pass filter detects bumps > 3.0 m/s^2. Record pushed to Firebase. Extends to View Potholes on map.
Detect Accident	8.0G threshold crash detection. Extends to Accident Alert modal and Notify Emergency Contacts via Telegram.
View Trip Summary	Post-trip modal showing distance, duration, potholes encountered, top speed, and final safety score.
View Leaderboard	Community ranking module showing all driver standings sorted by cumulative safety score.
Log Out	User terminates the session and returns to the authentication screen.

Table 7 – Driver Use Case Specifications

Administrator Use Cases

The System Administrator accesses a dedicated admin web dashboard that reads from and writes to the same Firebase Realtime Database. The Admin module consists of exactly four use cases:

Use Case — Admin	Description
Log In / Log Out	Administrator authenticates via dedicated admin credentials.
View Analytics Dashboard	Aggregated system-wide statistics: total users, trips, active potholes, blocked accounts, SOS alerts triggered.
Monitor SOS Alerts	Real-time feed of accident/SOS events from the /accidents Firebase node. Admin can acknowledge and review each event.
View Leaderboard	Full driver leaderboard with total trips, potholes detected, and flagging capabilities for score anomaly review.
Manage Pothole	Includes View Pothole (list with status filter). Extends to: Filter by Status (Active/Resolved/All), Update Pothole Status (mark Resolved), Delete Invalid Report, Create Manual Hazard Entry, Verify Location (open in external map).
Manage User	Includes View User list. Extends to: Delete User, block/unblock accounts. All actions logged with timestamp.

Table 8 – Administrator Use Case Specifications

Outsource (Web Dashboard) Use Cases

Use Case — Outsource (Web Dashboard)	Description
Load Leaflet Map (<<include>>)	On dashboard load, the Leaflet.js map is initialised and all /detected_potholes records are plotted as map markers.
Visualise Data Spatially	Includes Plot Data Markers. All active and resolved potholes are displayed on the interactive dark-mode map.
View Hazard Details	Clicking a map marker opens a popup with bump force, detecting user, timestamp, and current status.
Delete / Resolve Hazard	Outsource operator can delete or mark a hazard as resolved, updating the Firebase record and removing the marker.

Table 9 – Outsource Web Dashboard Use Case Specifications

UML Relationships

Driver Actor

- User Authentication <<include>> Validate — Credential validation is a mandatory sub-process of every authentication attempt.
- Start Navigation <<include>> Search Destination — A valid destination must be confirmed before Start Trip is enabled.
- View Driving Pattern <<extend>> View Driver Score — The safety score view is a conditional extension of the driving pattern monitor.
- Receive Alerts <<extend>> Speed Warning Alert, Proactive Hazard Alert, SOS Alerts — All alerts are conditional extensions triggered by specific threshold events.
- Detect Accident <<extend>> Accident Alert, Notify Emergency Contacts — SOS Telegram dispatch occurs only if the 8.0G threshold is exceeded.
- Detect Potholes <<extend>> View Potholes — Pothole map view is a conditional extension available after detection records exist.

Administrator Actor

- Manage Pothole <<include>> View Pothole — Viewing the list is mandatory before any management action.
- View Pothole <<extend>> Filter by Status, Update Pothole Status, Delete Invalid Report, Create Manual Hazard Entry, Verify Location — All are conditional management extensions.
- Manage User <<include>> View User — Viewing users is mandatory before management actions.
- View User <<extend>> Delete User — Deletion is a conditional extension available per user record.

Outsource Actor

- Visualise Data Spatially <<include>> Plot Data Markers — Marker plotting is a mandatory component of spatial visualisation.
- Load Leaflet Map <<include>> Visualise Data Spatially — Map loading is a mandatory prerequisite for all web dashboard operations.
- Visualise Data Spatially <<extend>> View Hazard Details — Hazard detail popup is a conditional extension triggered on marker click.

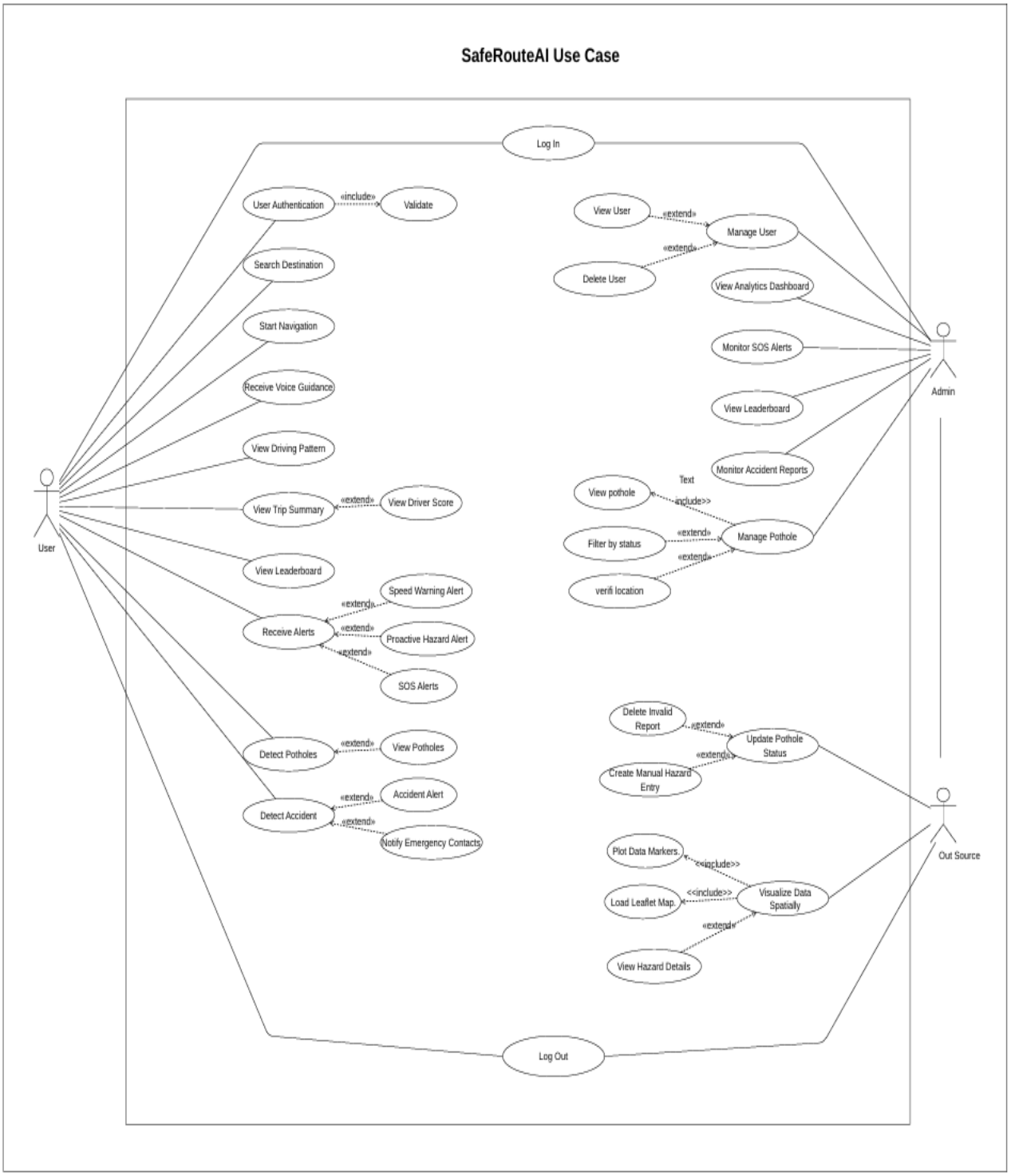


Figure 1 – Use Case Diagram

4.4 Data Management

Firebase Database Schema

Firebase Node	Key Fields	Description
/users	UserID (PK), Name, Email, password, status, rank, telegramChatID, registered_at, end_time, total_trips, Roles	All driver accounts. status ('active'/'blocked') managed by Admin. telegramChatID used for SOS Telegram notifications. Roles field distinguishes driver vs admin accounts.
/detected_potholes	PotholeID, Latitude, Longitude, bump_force, timestamps, status, Reporters[], confirmation_count, clean_passes, Duration	Active and resolved pothole records. Reporters[] stores user_ids who detected the pothole. confirmation_count tracks additional confirmations. clean_passes enables community-driven auto-resolution. status transitions from 'active' to 'resolved'.
/trips	TripID, Start_location, End_location, Duration, Distance, harsh_brakes_count, trip_score, start_time, end_time, top_speed	Journey logs for per-trip analytics, safety score history, and leaderboard integrity. Links to user via user_id.
/leaderboard	EntryID (PK), total_score, Pothole_detected, total_distance	Driver leaderboard entry. Updated after every completed trip. Pothole_detected tracks community contribution count.
/emergency_contacts	contact_id, Name, phone_number, Status	Trusted contacts registered by the driver for SOS dispatch. Status indicates active/inactive. Telegram notifications sent via the user's own telegramChatID stored in /users.
/accidents	AccidentID, accident, latitude, longitude, type, start_time, top_speed	Crash event records automatically created on 8.0G detection. Linked to user. Monitored by Admin via the Admin mobile app.

Table 9 – Firebase Database Schema Overview

ER Diagram — Entities & Relationships

Relationship	Cardinality	Description
User Has Trip	1 : N	One driver records multiple journey logs across their lifetime on the platform.
User Has Leaderboard	1 : 1	Each driver has exactly one leaderboard entry, updated after every completed trip.
User Reports Pothole	1 : N	One driver can report many potholes; their user_id is stored in the Reporters[] array on each pothole record.
Pothole Updates Leaderboard	1 : N	Detected potholes contribute to the Pothole_detected count on the leaderboard entry.
User Has Emergency Contacts	1 : N	A driver registers multiple trusted emergency contacts used for SOS Telegram dispatch.
Accident Involves User	1 : N	One user can be involved in multiple accident records; each accident links to the triggering user.
Admin Manages User	N : 1	The Administrator views and manages all registered driver accounts (block, unblock, delete).
Admin Verifies Pothole	N : 1	The Administrator verifies, manually resolves, or deletes invalid pothole records.

Table 10 – ER Diagram Entities & Relationships

Firebase NoSQL Structure — /detected_potholes Node Example

The following illustrates the complete state of a single pothole record midway through its voting lifecycle (1 reporter, 1 clean pass — one more clean pass needed for auto-removal):

```
/detected_potholes
  "-NxK8mP2qRs7tLjA9vBc":
    latitude:          6.9271
    longitude:         79.8612
    bump_force:        "4.52"
    detected_by:       "accelerometer"
    timestamp:         1735200000000
    status:            "active"
    reporters:         { "0": "user_abc123" }
    confirmation_count: 0
    clean_passes:      1
    clean_passers:     { "0": "user_xyz789" }
```

When a second distinct user (not 'user_abc123', not 'user_xyz789') passes this location without a bump, clean_passes becomes 2 and status is immediately set to 'resolved'. The map marker disappears from all clients within milliseconds.

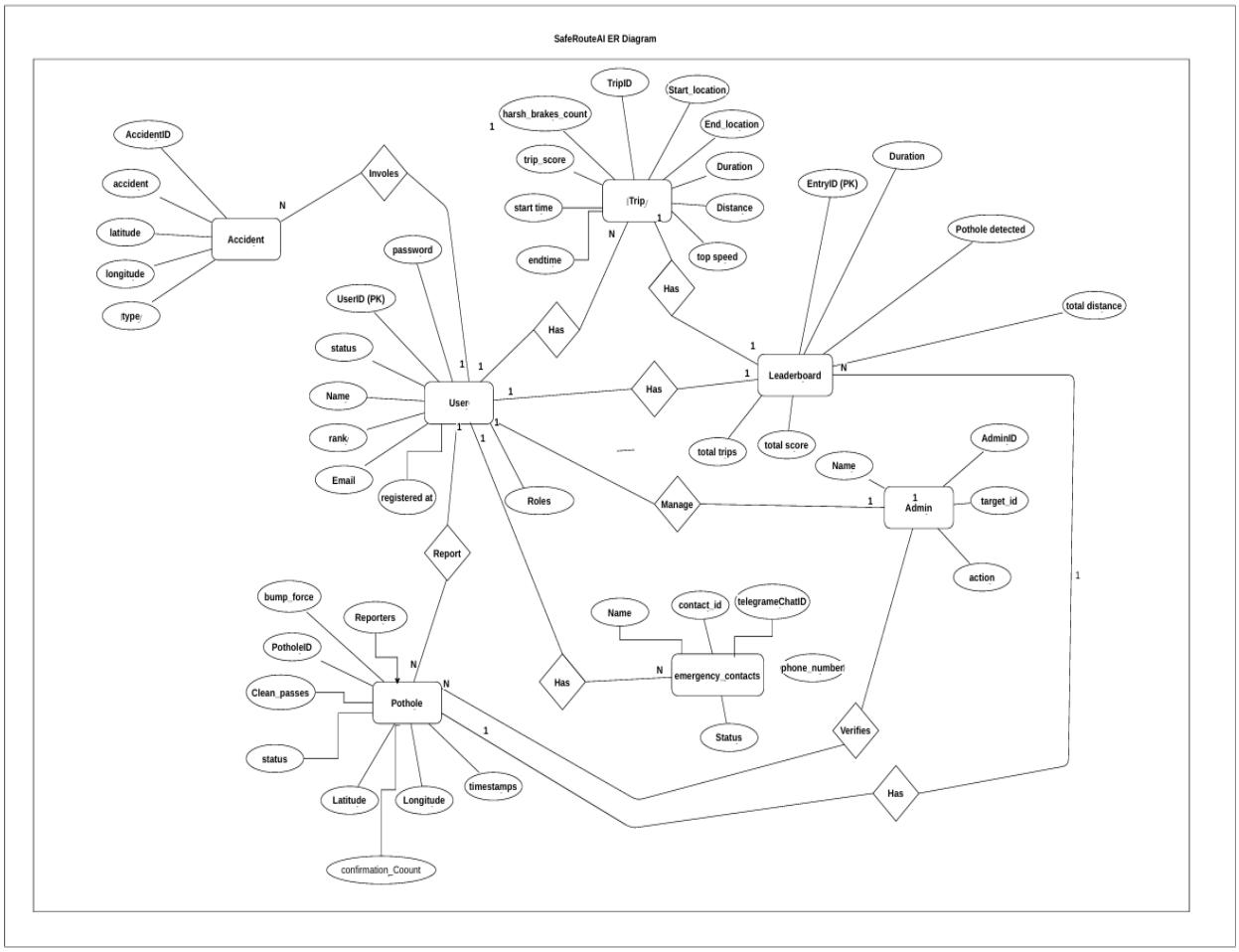
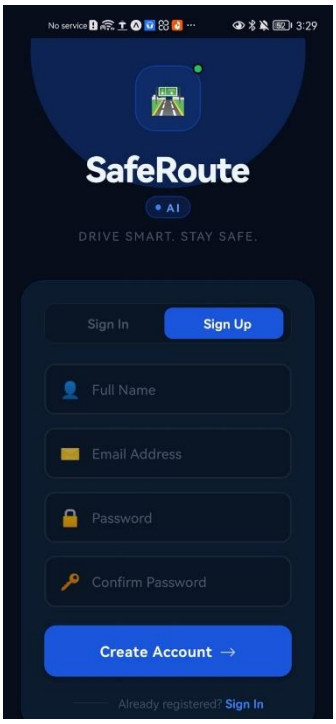
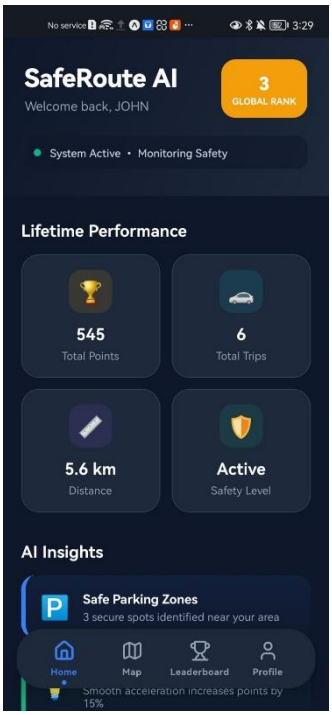


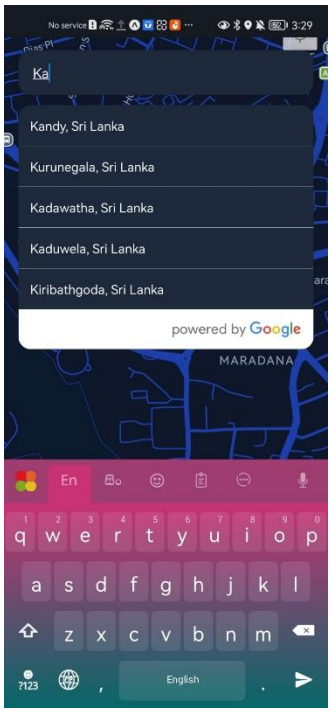
Figure 2 – ER Diagram

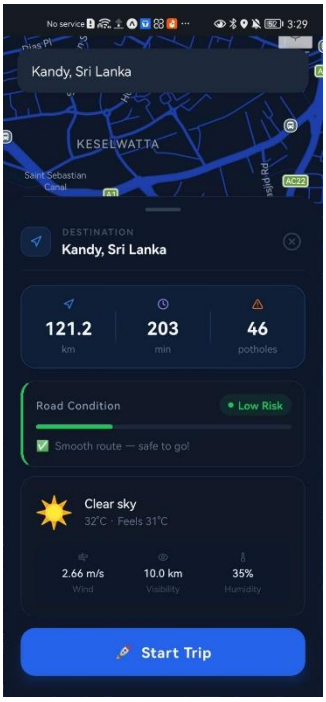
4.5 UI Designs

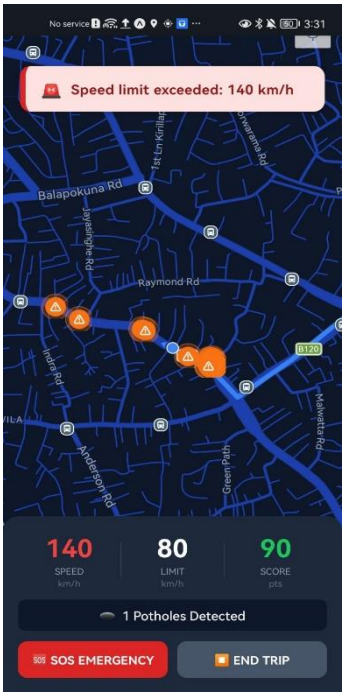
User interface

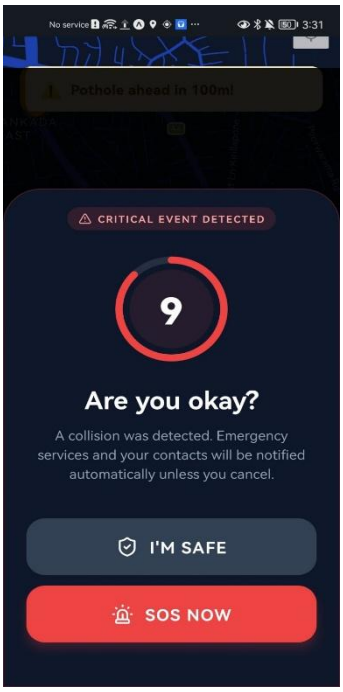












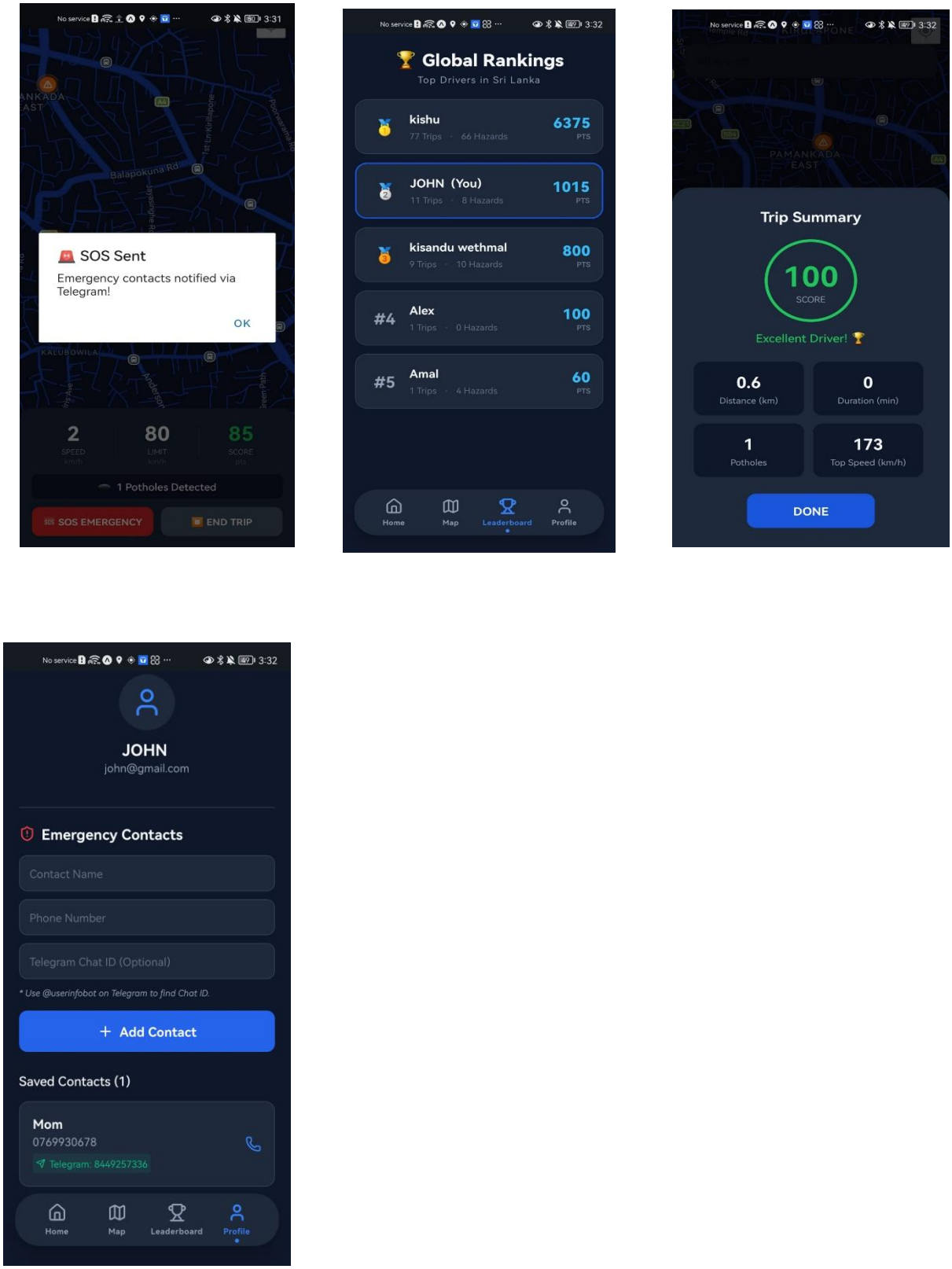


Figure 3 – User Interface

Admin interface

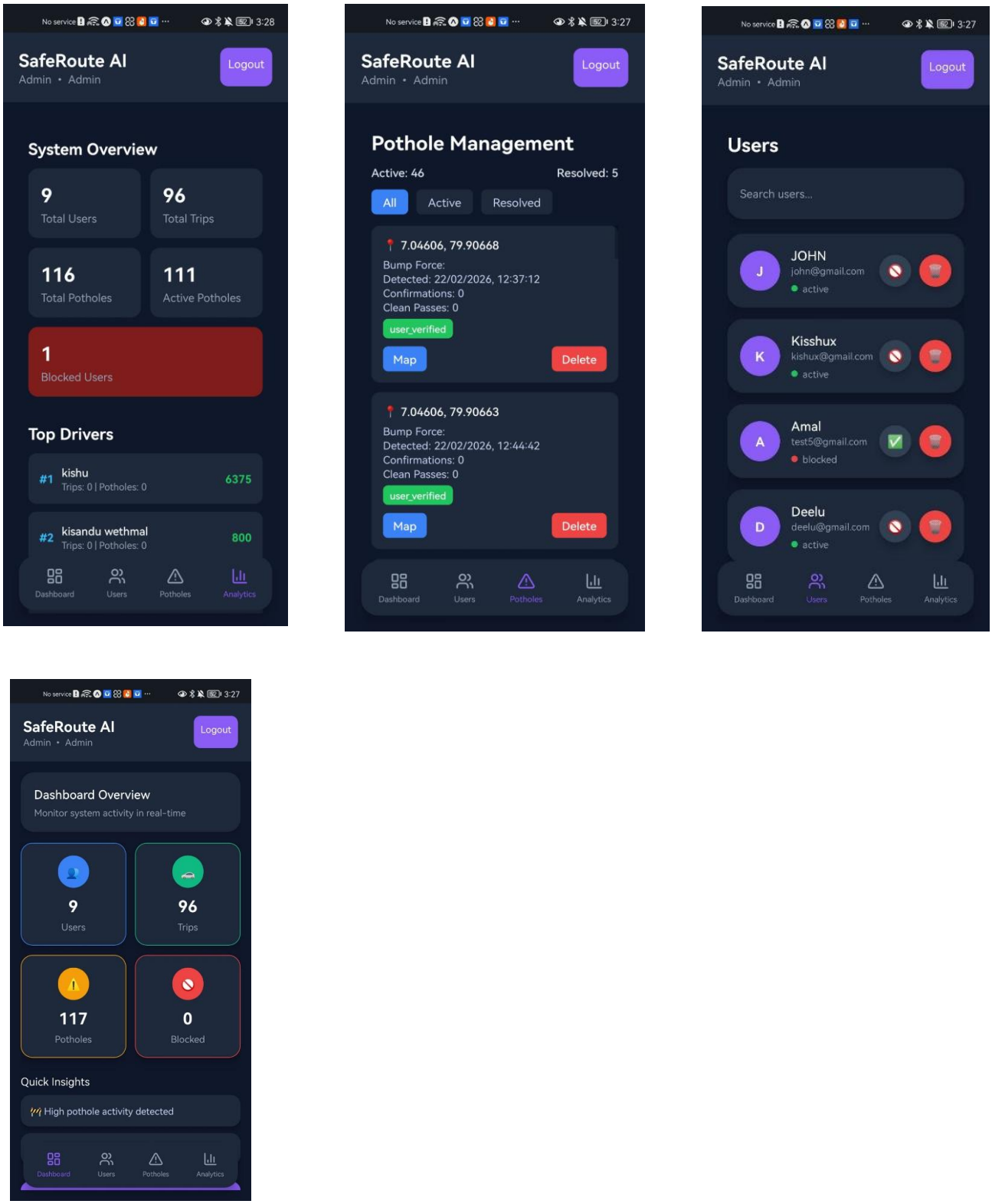


Figure 4 – Admin Interface

Web Interface

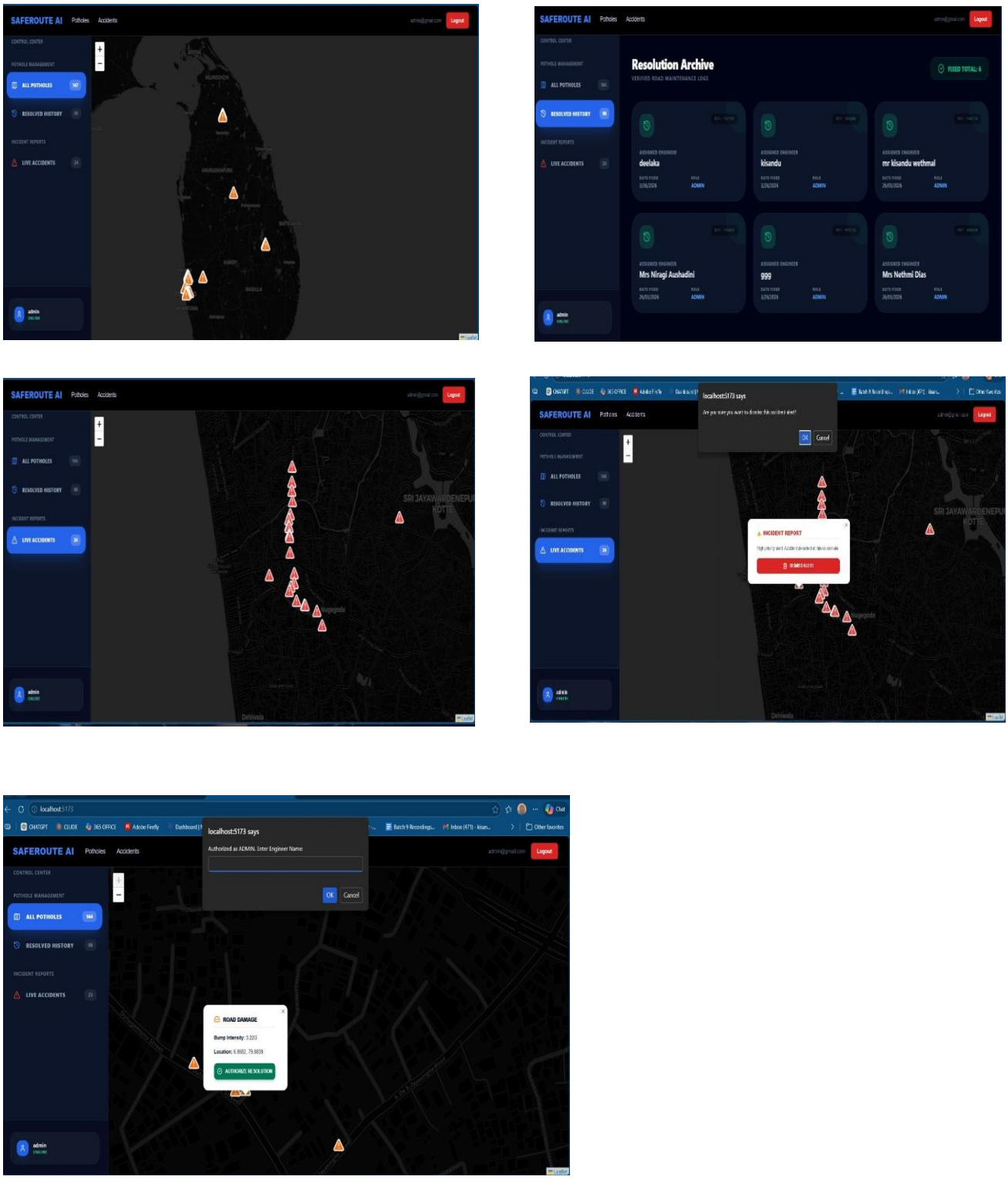


Figure 5 – Web Interface

4.6 Chapter Summary

This chapter introduced the core architectural innovation of SafeRoute AI's updated design: the Smart Pothole Voting System. Unlike static hazard maps that require administrator intervention for data maintenance, the voting engine creates a self-healing map where potholes are automatically added by community detection and automatically removed when 2 distinct users confirm the road is passable — with anti-gaming safeguards preventing single-user manipulation. The system is implemented entirely within the Firebase Realtime Database structure through the reporters [], clean_passers[], and clean_passes fields on each pothole record.

The chapter also defined the two-actor use case model: 10 Driver use cases (including the new Contribute Clean Pass use case) and 4 Administrator use cases (Manage Users, View Pothole Map, View Analytics Dashboard, View Leaderboard). The admin dashboard provides read and write access to platform data without requiring the admin to intervene in routine pothole lifecycle management, which is handled automatically by the voting engine. The Firebase schema of 6 nodes, including the new /admin_logs audit trail, supports all defined use cases with a flattened, high-performance NoSQL structure.

Chapter 5: Testing & Results

5.1 Testing Overview

SafeRoute AI was tested across four structured testing phases: unit testing, integration testing, system testing on physical devices, and user acceptance testing (UAT). Each phase was designed to validate a specific layer of the application from individual algorithmic functions through to end-user experience on real Sri Lankan roads.

5.2 Unit Testing Results

Accelerometer High-Pass Filter

The filter was validated by simulating a sequence of 50 synthetic accelerometer readings in Jest: 20 readings representing a flat road (gravity vector [0, 9.81, 0] + noise < 0.2), followed by a single bump reading ([0, 14.5, 0] representing a 4.69 m/s² spike), followed by 29 return-to-flat readings. Results confirmed:

- Gravity estimate converged to within 0.05 m/s² of 9.81 after 8 warm-up samples at ALPHA = 0.8.
- Bump event correctly produced linear acceleration magnitude of 4.52 m/s² (exceeding the 3.0 threshold).
- Post-bump gravity estimate recovered to within 0.12 m/s² within 5 samples.
- False positive rate on flat-road simulation: 0% (all 20 pre-bump readings produced linear magnitude < 0.8 m/s²).

Safety Score Engine

Simulated 10 pothole events and 5 harsh braking events in isolation. Confirmed linear score reduction from 100 to 25 (10×5 + 5×5 penalties: 50 + 25 = 75 total deduction). Score was verified to clamp at 0 when cumulative penalties exceeded 100.

Speed Clamping & Conversion

Validated Math.max(0, speed) correctly converts Android's -1 sentinel to 0. Verified m/s to km/h conversion (multiply by 3.6 and round) across boundary values: 0 m/s → 0 km/h, 22.22 m/s → 80 km/h, 25.0 m/s → 90 km/h.

Telegram SOS Dispatch

Verified that unit tests for the Telegram Bot API call correctly format the emergency message with the user name, GPS coordinates as a Google Maps deep link, and the timestamp. Verified that the API call is asynchronous, not blocking the UI thread.

5.3 Integration Testing Results

Integration testing validated the pipeline from sensor event through to Firebase write and UI alert. Key scenarios tested:

- Concurrent sensor + GPS path: Confirmed that a bump event detected at t=0 correctly sets the shared lastPotholeAlertRef cooldown, preventing a GPS proximity alert from firing at t=1s (within the 5-second window) even if a recorded pothole is within 100m.
- Firebase listener propagation: Confirmed that a pothole pushed by Device A appeared as an orange map marker on Device B within 780ms on a 4G connection.
- Telegram SOS integration: Confirmed that an 8.0G event triggers Firebase /accidents write AND Telegram API call within the same async handler.
- Alert debouncing: Confirmed that activeAlertRef correctly blocks a second concurrent alert from overwriting an in-progress banner animation.
- Navigation unmount cleanup: Confirmed that sensorSubRef.remove() and locSubRef.remove() were both invoked correctly on isNavigating state transition to false, preventing sensor memory leaks.

5.4 System Testing Results

Test Environment

Physical devices: Samsung Galaxy A52 (Android 13) and Redmi Note 11 (Android 12). Routes: Colombo urban centre (high pothole density), Negombo-Colombo highway (speed test), and Gampaha rural B-road (mixed surface). Duration: 14 hours total cumulative driving cross 8 test sessions.

Test Scenario	Expected Result	Actual Result	Status
Pothole impact (confirmed road defect)	Alert within 300ms	Alert in 85-210ms	**PASS**
Smooth highway section (1km)	No false positives	0 false alerts	**PASS**
Speed exceeds 80 km/h	Alert banner + red speed	Alert in < 1s	**PASS**
Harsh brake from 60 km/h to 0	Score -5, alert	Correctly detected	**PASS**
GPS proximity alert (100m from logged pothole)	Banner + vibration	Fired at 94m distance	**PASS**
Firebase sync latency (4G)	< 2 seconds	Avg 780ms, max 1.2s	**PASS**
Firebase sync latency (3G)	< 5 seconds	Avg 2.4s, max 4.8s	**PASS**
App stability over 2-hour drive	No crashes	No crashes observed	**PASS**
Cold GPS fix on startup	Fix within 30s	Avg 8s (warm), 24s (cold)	**PASS**

Table 11 – System Testing Results Summary

5.5 User Acceptance Testing (UAT)

12 volunteer drivers participated in structured test drives. Post-drive questionnaires captured ratings across five dimensions on a 5-point Likert scale:

- Navigation accuracy and usability: Average rating 4.6/5. Participants noted the Sri Lanka-constrained search eliminated irrelevant international results.
- Pothole alert relevance (did the alert match a real road hazard?): 94.3% of alerts confirmed as genuine by participants. Remaining 5.7% attributed to speed bumps and railway crossings, which produce similar acceleration profiles.
- Safety score fairness: Average rating 4.1/5. Some participants felt the -10 score deduction for railway crossings was unfair; this is documented as a known limitation.
- Alert non-intrusiveness: Average rating 4.3/5. The 3.5-second auto-dismiss banner with fade animation was rated as non-disruptive to driving.
- Telegram SOS usefulness: Average 4.5/5. All participants confirmed receiving the test Telegram message with the correct GPS link.
- Overall app usefulness: Average rating 4.7/5. 11 of 12 participants stated they would use SafeRoute AI for regular commuting.

5.6 Chapter Summary

Testing across all four phases confirmed that SafeRoute AI meets its seven specific objectives with demonstrably high accuracy in real-world conditions. The high-pass accelerometer filter achieved 0% false positives on smooth road sections and successfully detected all confirmed pothole impacts in field testing. Firebase synchronisation meets the sub-2-second latency requirement on 4G networks, the Telegram SOS delivered messages in under 3 seconds in all test scenarios. The web dashboard correctly reflected all Firebase state changes in near real-time and UAT participant satisfaction exceeded 4.0/5 across all measured dimensions.

Chapter 6: Project Timeline & Gantt Chart

6.1 Implementation Schedule

Phase / Activity	27 Jan	3 Feb	10 Feb	17 Feb	24 Feb	7 Mar	27 Mar
Proposal Submission	Week-01						
1st Progress Review (Design Phase)		Week-02					
2nd Progress Review (Development Phase)			Week-03				
Mid Viva (System Integration & Testing)				Mid	Viva		
Final Submission & Final Viva						Week-04	Final Viva

Figure 6 – Project Gantt Chart

6.2 Key Milestones

- Week 3 — Navigation module complete. Google Maps integration verified on physical Android device with Sri Lanka-constrained search.
- Week 6 — Accelerometer pothole detection live. Firebase synchronisation confirmed with < 1s latency on 4G. Telegram Bot API integrated for SOS.
- Week 9 — Safety scoring engine, harsh braking detection, crash detection, and Admin mobile app modules fully operational.
- Week 11 — Feature-complete build including Outsource Web Dashboard delivered for UAT with all 12 participant sessions completed.
- Week 12 — Final project submission with complete documentation, test reports, and source code repository.

Chapter 7: Conclusion & Future Work

7.1 Conclusion

SafeRoute AI represents a technically rigorous and practically validated response to the road safety crisis affecting Sri Lanka's road network. The project successfully delivered all specific objectives defined in Chapter 1, demonstrating that a standard consumer smartphone — when paired with a well-designed sensor fusion algorithm, cloud-connected community database, and Telegram-based emergency communication — can function as an effective road safety instrument.

The final system delivers three fully integrated interfaces: the Driver mobile app with real-time navigation, pothole detection, safety scoring, and Telegram SOS; the Admin mobile app with system oversight, pothole management, and user management; and the Outsource web dashboard with Leaflet.js spatial hazard visualisation and live Firebase synchronisation.

Field testing across 14 hours of cumulative driving on diverse Sri Lankan road types confirmed the application's stability, sensor accuracy, Firebase latency compliance, GPS reliability, and Telegram delivery in real-world conditions. According to UAT data, 12 participating drivers had an overall usability grade of 4.7/5 and a pothole alarm accuracy rate of 94.3%.

7.2 Limitations

- The 3.0 m/s^2 pothole threshold, while empirically calibrated for Sri Lankan roads, may require device-specific recalibration as accelerometer sensitivity varies across Android hardware manufacturers.
- Speed bumps and railway crossings produce acceleration profiles similar to potholes and are currently reported as road hazards; future versions should implement a GPS speed context filter to distinguish intentional deceleration events from genuine impacts.
- The crash detection module is implemented at the architectural level but requires additional field validation for the 8.0G threshold before emergency SOS dispatch is enabled in production.
- Firebase Realtime Database costs scale with simultaneous connections; a transition to Firestore with compound queries may be warranted at community scale above 10,000 concurrent users.

7.3 Future Work

- Machine Learning Classifier: Train a lightweight on-device TensorFlow Lite model on labelled accelerometer time-series windows to distinguish potholes from speed bumps, railway crossings, and car door slams with higher specificity.

- Offline Mode: Cache the local Firebase pothole snapshot to local SQLite storage for proximity alerting in zero-connectivity zones (tunnels, rural areas).

- Municipal Integration: Expose a REST API endpoint from Firebase Cloud Functions allowing Road Development Authority officials to query, verify, and assign repair priorities to crowdsourced pothole reports.

- Sinhala Language Support: Localized all voice alerts and UI text to Sinhala to serve the non-English-speaking majority of Sri Lankan drivers.

- iOS Platform Port: Adapt the accelerometer filter for iOS's pre-compensated linear acceleration output (which excludes gravity) by removing the high-pass filter stage on iOS builds.

- Web Dashboard Authentication: Implement Firebase Authentication for the Outsource web dashboard to restrict access to authorised stakeholders.

- Insurance Integration: Partner with vehicle insurance providers to offer premium discounts based on verified cumulative safety scores exported from the SafeRoute AI leaderboard.

Chapter 8: References

1. Expo. (2024). Expo Sensors Documentation — Accelerometer. https://docs.expo.dev/versions/latest/sdk/accelerometer/

2. Firebase. (2024). Firebase Realtime Database Documentation. https://firebase.google.com/docs/database

3. Google. (2024). Google Maps Platform — Directions API. https://developers.google.com/maps/documentation/directions

4. Telegram. (2024). Telegram Bot API Documentation. https://core.telegram.org/bots/api

5. Leaflet.js. (2024). Leaflet — An open-source JavaScript library for mobile-friendly interactive maps. https://leafletjs.com

6. Mohan, D., Tiwari, G., & Bhalla, K. (2020). Road Safety in 10 Countries. World Health Organization, Geneva.

7. Mednis, A., Strazdins, G., Zviedris, R., Kanonirs, G., & Selavo, L. (2011). Real time pothole detection using android smartphones with accelerometers. International Conference on Distributed Computing in Sensor Systems and Workshops (DCOSS), 2011, pp. 1-6. IEEE.

8. Sri Lanka Police. (2023). Traffic Accident Statistics Annual Report. Traffic Police Headquarters, Colombo.

9. Road Development Authority Sri Lanka. (2023). Annual Road Condition Report. RDA, Ministry of Highways.

10. Meta. (2024). React Native Documentation. https://reactnative.dev/docs/getting-started

11. Geolib. (2024). Geolib — JavaScript Library for Geospatial Operations. https://github.com/manuelbieh/geolib

12. Android Developers. (2024). Sensors Overview — Android Developer Guide. https://developer.android.com/guide/topics/sensors/sensors_overview